Midterm 02: True Random Number Generators (TRNGs)

Group E: Jack, Jesse, Omar, Thu Ta
ECE 4301 – Crypto Algo

1 Introduction

Most software uses pseudorandom number generators (PRNGs), which produce deterministic sequences based on an initial seed. This behavior is often acceptable for simulation or simple randomized algorithms. Cryptographic applications are different. In that setting, an attacker who can guess or reconstruct the seed can also predict all future outputs of the PRNG.

To avoid this problem, secure systems rely on true random number generators (TRNGs). A TRNG derives randomness from a physical noise source instead of a deterministic algorithm. This randomness is used to seed PRNGs, generate secret keys, nonces, and other cryptographic values that must be unpredictable.

This report describes several common types of TRNGs, explains the physical principles that make them unpredictable, and then relates these concepts to the hardware TRNG available on the Raspberry Pi. Finally, it summarizes my implementation of a Rust based TRNG analyzer that reads from the Raspberry Pi hardware source and performs basic statistical checks.

2 Types of TRNGs

2.1 Thermal noise TRNGs

Thermal noise, also called Johnson Nyquist noise, appears as small voltage fluctuations across a resistor due to the random motion of charge carriers. A thermal noise TRNG amplifies this noise and samples it with an analog to digital converter (ADC). The sampled bits are then processed to remove any bias.

The physical principle is random motion of electrons driven by temperature. Advantages include simplicity and the ability to implement the circuit with standard analog components. The main challenges are that the raw signal is very small and must be amplified, filtered, and digitized carefully in order to achieve a stable and unbiased entropy source.

2.2 Avalanche diode TRNGs

A reverse biased diode operated in the avalanche region exhibits random breakdown events. The timing and amplitude of these events are influenced by microscopic variations and quantum effects in the semiconductor. By observing the current or voltage fluctuations and digitizing them, the circuit can generate random bits.

The underlying principle is impact ionization inside the diode. Avalanche based TRNGs can provide high entropy density in a small area. On the other hand, they are sensitive to temperature and supply voltage, and they often require post processing such as whitening or hashing to remove bias and correlations.

2.3 Metastability based TRNGs

Digital circuits can be forced into metastable states, for example by racing two signals into a latch or flip flop at nearly the same time. In a metastable state the output hovers between logical zero and one for an unpredictable amount of time until small thermal fluctuations push it to one side.

Metastability based TRNGs use this effect to generate random bits. The entropy comes from the unpredictable resolution of the metastable state. These designs are attractive because they can be implemented with standard digital cells and integrated on chip. The main difficulty is to avoid systematic bias due to asymmetry in the circuit or timing.

2.4 Ring oscillator TRNGs

A ring oscillator consists of an odd number of inverters connected in a loop. The signal propagates around the loop and oscillates at a frequency determined by the inverter delays. In practice, the delay varies randomly due to thermal noise, supply noise, and device level variations. This introduces jitter in the phase of the oscillation.

Ring oscillator TRNGs typically use multiple oscillators and sample them with a reference clock. The sampled bits depend on the instantaneous phase of the oscillators, which is influenced by jitter. The design is compact and digital friendly, and is often used inside system on chip devices. To achieve high quality entropy, designers usually combine several oscillators and apply post processing.

2.5 Quantum TRNGs

Quantum TRNGs directly rely on quantum mechanical randomness, such as whether a photon passes through or is reflected by a beam splitter, or the time of arrival of single photons at a detector. The outcomes are fundamentally unpredictable according to quantum theory.

These TRNGs can provide very strong theoretical guarantees about randomness quality. However, they tend to require optical components or radiation sources, which makes them less suitable for low cost embedded systems like a Raspberry Pi.

3 Raspberry Pi Hardware TRNG

Modern Raspberry Pi system on chip devices expose a hardware random number generator through the Linux device file `/dev/hwrng`. Internally, the chip includes a physical entropy source, such as ring oscillators or amplified noise, along with some form of conditioning logic that processes the raw signal into uniformly distributed bits.

The operating system provides a kernel driver that reads from the hardware source, performs basic health tests, and then offers the resulting bytes through `/dev/hwrng`. User space programs can read from this device as if it were a file. The hardware TRNG is then used to seed the kernel random pool and also to supply randomness directly to applications.

Although the exact internal design of the Raspberry Pi TRNG is not fully documented, its behavior is consistent with a ring oscillator or noise based design that is conditioned by a whitening function or cryptographic primitive.

4 Implementation on the Raspberry Pi

For this project I implemented a TRNG demo in Rust that runs on a Raspberry Pi and reads directly from `/dev/hwrng`. The program performs the following steps:

1. Open `/dev/hwrng` and read 100,000 bytes of random data.
2. Compute the mean and standard deviation of the byte values in the range 0 to 255.
3. Build a histogram with 256 bins, one for each possible byte value.
4. Compute the Shannon entropy per byte from the histogram.
5. Count the total number of ones and zeros at the bit level and compute the fraction of ones.
6. Save the histogram as a CSV file, which can be plotted using a small Python script.

The Rust code uses standard library I/O to read from the device file and simple loops to compute the statistics. A separate Python script reads the CSV file and generates a bar plot of the histogram, saved as a PNG image. This image can be included in the project repository and in the written report as evidence of the distribution shape.

5 Results

A typical run of the Rust TRNG analyzer produced the following results:

- Mean byte value: approximately 127.64, where a perfectly uniform distribution on 0 to 255 would have mean 127.5.
- Standard deviation: approximately 73.94, close to the theoretical value for a uniform distribution on 0 to 255.
- Shannon entropy per byte: approximately 7.998 bits, very close to the ideal value of 8.0 bits.
- Bit balance: a ones fraction of about 0.5009, which is very close to the ideal 0.5.

The histogram of the 256 byte values is visually flat, with each value appearing roughly the same number of times. These numerical and visual checks indicate that the data from `/dev/hwrng` is very close to uniformly distributed over bytes and that there is no obvious bias toward zero or one at the bit level.

In practice, a complete validation of a TRNG would use a larger test suite such as NIST SP 800-22 or Dieharder. However, for the scope of this assignment, the basic statistics described above are sufficient to demonstrate that the Raspberry Pi hardware TRNG behaves like a high entropy source.

6 Conclusion

This report reviewed several types of true random number generators, including thermal noise based designs, avalanche diode TRNGs, metastability based designs, ring oscillator TRNGs, and quantum TRNGs. Each relies on a different physical mechanism, but they share the goal of producing unpredictable bits suitable for cryptographic use.

On the Raspberry Pi, the system on chip provides a hardware TRNG that is accessible through the `/dev/hwrng` interface. I implemented a Rust based analyzer that reads from this device, computes simple statistics, and exports a histogram. The measured mean, variance, entropy, and bit balance are all consistent with a high quality random source.

These results show that the Raspberry Pi hardware TRNG is appropriate as an entropy source for seeding PRNGs and for generating cryptographic keys in applications that run on the platform.

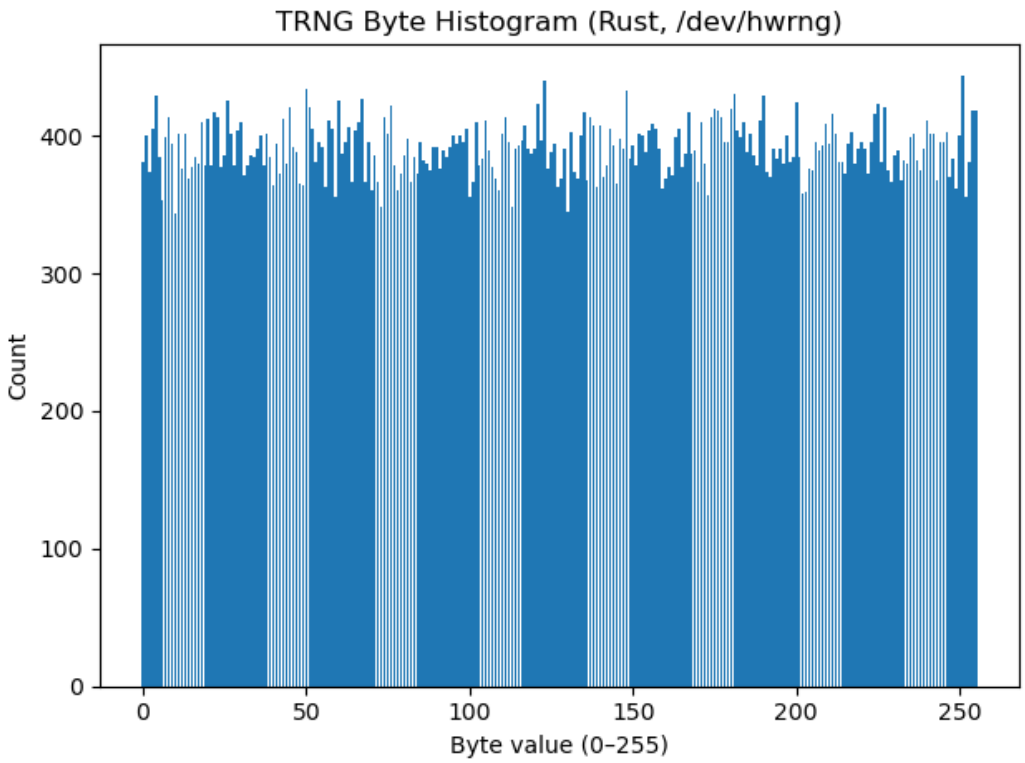

Figure 1: Byte frequency histogram of 100,000 samples from the Raspberry Pi hardware TRNG using Rust. The distribution is visually flat, which indicates that each byte value from 0 to 255 appears with roughly equal likelihood.